

设计报告

简易自行瞄准装置

摘要

本装置设计实现了一种基于 MSPM0G3507 微控制器的简易自行瞄准装置，由自动寻迹小车和激光瞄准模块组成。系统采用双电源独立供电方案，自动寻迹小车通过五路灰度传感器进行循线，采集角度值与平衡位置进行比较，配合 PID 算法控制编码电机，实现自动寻迹，利用二维步进电机云台控制 405nm 蓝紫激光笔进行精确瞄准。针对靶面定位问题，提出基于几何位置解算的实时瞄准算法，在静态和动态条件下均能实现高精度瞄准。本系统实现了具有自动寻迹小车和激光瞄准的简易自行瞄准装置，包括自动寻迹功能和自动激光寻靶等功能。

关键字：MSPM0G3507;激光瞄准;自适应 PID 控制;二维步进电机云台;计算机视觉

目 录

第一章 系统方案	2
1.1 TIMSPM0 系列单片机的论证与选择	2
1.2 嵌入式视觉平台的方案与选择	2
1.3 云台电机的论证方案与选择	2
1.4 角度传感器的论证方案与选择	3
1.5 总体方案描述	3
第二章 理论分析与计算	4
2.1 循迹小车自动行驶控制	4
2.2 同步画圆控制策略	4
第三章 电路与程序设计	4
3.1 系统架构	4
3.2 步进电机云台系统框图设计	5
3.3 程序设计	5
第四章 测试方案与测试结果	5
4.1 测试方案	5
4.2 测试条件与仪器	6
4.3 测试结果与分析	6
参考文献	8
附录	9

第 1 章 系统方案

1.1 TIMSPM0 系列单片机的论证与选择

TIMSPM0 系列微控制器凭借其高性能、超低功耗与丰富的外设集成，在工业控制、消费电子及智能传感器等领域广受认可。以 MSPM0G3507 为例，该芯片专为高精度实时控制场景，运行功耗低至 96 μ A/MHz 达到高性能与低功耗的极致平衡，并且支持多协议协同与低功耗唤醒，简化工业总线互联设计有着高度集成的外设资源 TI 提供 SysConfig 图形化配置工具，可直观管理引脚复用与外设冲突，搭配 Code Composer Studio™ IDE，大幅缩短开发周期开发效率显著提升。TI 的 MCU 在处理复杂算法时展现出卓越的效率，这对于实现精确的寻迹控制来说至关重要，进一步提升了其在智能控制应用中的优势。

1.2 嵌入式视觉平台的方案与选择

方案一：庐山派 K230 计算机视觉平台

作为配备双核 RISC-V 玄铁 C908 处理器的边缘 AI 开发平台，庐山派 K230 第三代 KPU 提供 6TOPS 的超级计算能力（性能是 K210 的 13.7 倍），结合 Python-first CanMV 开发框架，实现高效部署，实时性出色。与 MSPM0 控制框架的无缝兼容，以及有效处理瞄准模块的快速响应要求。

方案二：采用 OpenMV 嵌入式计算机视觉平台

OpenMV 是一个基于 ARM 单片机的嵌入式视觉开发平台，以 MicroPython 为核心编程语言。其核心优势在于开发门槛低，开发人员可以通过 Python 快速实现基本的图像处理。该平台提供了丰富的图像处理和传感器接口库，适合教学实验和轻量级视觉原型开发。

总结：如上可知 K230 具有更强的实时控制兼容性，低延迟性能，具备 40 个 FPIOA 可以自由配置。确保性能部分（如行驶过程中连续激光发射）满足同步和可靠性要求。故选择方案一。

1.3 云台电机的方案与选择

方案一：步进电机。具有高频响应与动态性能，能实现高频微动，匹配云台快速纠偏需求，低速下可达到很高扭矩，功耗略高。霍尔传感器反馈消除开环失步风险，能够

胜任高频精密任务。

方案二：舵机。舵机内置位置反馈，功耗低，可以实现精确位置控制但存在虚位，难以高频修正，精度需要妥协。

本任务中需要云台不断进行小角度调整，需要更高的精度来实现功能。基于上述分析，本系统采用方案一步进电机，作为瞄准系统的控制单元，能够高效地实现自动跟踪。

1.4 角度传感器方案与选择

方案一：微处理单元 MPU6050。芯片包含三轴加速度计及三轴陀螺仪，外围电路设计简单，采用 IIC 通信协议，具有量程大，精度高，功耗低，满足项目对角度精度的需求。

方案二：JY61P 姿态传感器，融合三轴加速度计与陀螺仪，通过卡尔曼滤波算法输出实时姿态角，精度达 $\pm 0.2^\circ$ ，支持 200Hz 高频回传，满足复杂路径下的快速响应需求即用性集成高大幅降低开发复杂度。

基于上述分析，JY61P 姿态传感器在精度、响应速度、抗干扰性及集成便利性上优于其他方案满足小车在指定路径上精确行驶和控制的需求故采用方案二。

1.5 总体方案描述

MSPM0G3507 控制编码器电机，四路寻迹模块。K230 视觉模块进行矩形识别的同时负责步进电机驱动板通信。步进电机驱动板采用 32 位 ARM Cortex-M0+ HC32 芯片控制。微控制器单片机控制 TB6612 直流电机驱动输出 PWM 波，配合 JY61P 姿态传感器得到的实际偏航角和加速度，控制小车自动行驶。步进电机云台搭配 K230 和激光笔，K230 识别定位目标靶位置，通过串口通信反馈到步进电机驱动板，通过 PID 算法对步进电机云台进行闭环控制，实现了目标靶的自动追踪功能。

第二章 理论分析与计算

2.1 循迹小车自动行驶控制

循迹小车通过五路灰度获取姿态数字量，换算为实际角度偏差后，经位置式 PID 计算输出 PWM，动态调整车身；编码器记录脉冲得实际速度，经增量式 PID 计算输出 PWM，闭环控制速度；两者串级关联实时运算，实现沿轨迹匀速行驶。

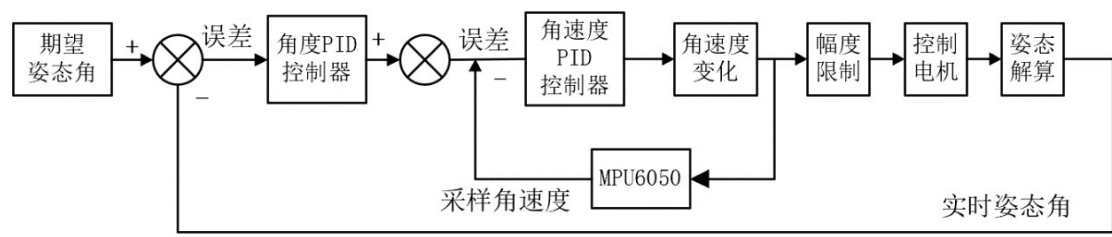


图 1 循迹小车自动行驶原理图

2.2 同步画圆控制策略

针对发挥部分要求(3)，设计角度-时间同步控制算法。设小车线速度 v ，轨迹周长 $C=400\text{cm}$ ，画圆周期 $T=C/v$ 。将目标点坐标 (x_t, y_t) 代入位置解算模型，实时计算云台角度。为确保同步精度，采用位置-速度双闭环控制，同步误差控制在 $T/8$ 以内。

第三章 电路与程序设计

3.1 系统架构

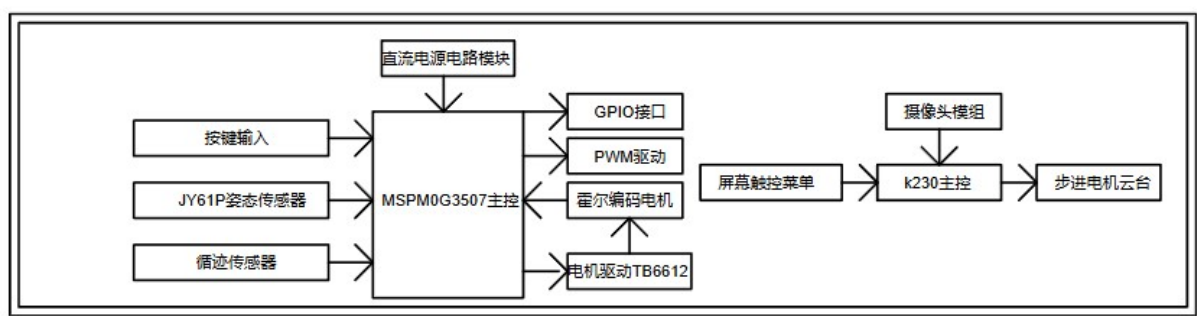


图 2 系统总体框图

3.2 程序设计

该程序以模块化设计为核心思路，结合 C 语言和 Python 开发，在 Keil5 环境调试实现视觉检测、电机控制等功能。主程序经初始化后，通过开关触发执行，分设三个功能模式：模式一聚焦小车自动寻迹，支持 1 ~ 5 圈切换，以 20cm/s 速度沿轨迹行驶，侧重电机控制与路径识别协同；模式二专注固定状态下的瞄准功能，由云台自动旋转追踪目标靶并发射激光，核心是视觉检测与云台驱动配合；模式三整合前两者，小车寻迹行驶 N 圈时，云台同步追踪靶面，控制激光沿 6cm 红色圆弧画圆，实现运动与瞄准的动态协同，各模块独立又互联，便于功能迭代与问题排查。

3.3 程序设计

软件程序设计采用模块化的结构，便于分析和实现功能，分如下三个功能。

模式一：小车可以沿行驶轨迹自动寻迹行驶，行驶圈数 N 可在为 1~5 之间切换，行驶速度为 20cm/s。

模式二：小车置于场地中固定不动，启动瞄准模块，云台自动旋转寻找追踪目标靶，并发射激光击中靶心。

模式三：启动小车和瞄准模块，沿轨迹自动寻迹行驶 N 圈，云台持续追踪目标靶，激光笔射向靶面，沿靶面上半径 6cm 的红色圆弧同步画圆。

第四章 测试方案与测试结果

4.1 测试方案

1、硬件测试：这一阶段的重点是验证各个硬件单元（如寻迹传感器、激光模块，步进电机、二维云台、电源开关和 LED、车载电池等）的独立功能是否齐全，并确保其性能指标达到预期标准。

2、程序测试：利用专业开发工具 CCS Theia、Keil5、CanMV 等，编写控制小车、云台控制的程序代码。通过在单片机上的反复运行和调试，发现并纠正了潜在的编程错误和逻辑漏洞。

3、软硬件集成调试：将设备放置在问题中指定的位置。根据竞赛规则或测试要求，观察并记录设备是否准确地完成了规定的任务（如沿着规定的路径循迹、激光打靶等）以及完成这些任务所需的时间。

4.2 测试条件与仪器

(1) 试验条件：多次检查，确保运行正常，参数正确。硬件电路和仿真电路必须与原理图完全一致，并进行精度检查。检查硬件电路，确保没有误焊。

(2) 检测仪器：秒表、直尺、UV 测试卡纸、打印现场地图。

4.3 测试结果与分析

1、测试结果

① 小车可以沿行驶轨迹自动寻迹行驶，行驶圈数 N 可在 $1\sim 5$ 之间设定，要求行驶时间 $t \leq 20s$ ，此时瞄准模块电源开关断开。

表 B-1 基础部分 1 测试结果

次数	1	2	3	4	5
时间 (s)	18.5	18.5	18.7	18.8	17.8

② 将小车放置在场地中，位置和姿态自定。启动瞄准模块在 2s 内发射激光击中靶心，要求光斑痕迹距靶心最大距离 $D1 \leq 2cm$ 。

表 B-2 基础部分 2 测试结果

次数	1	2	3	4	5
打靶用时 (s)	1.2	1.1	0.9	1.2	1.2

③ 将小车放置在行驶轨迹上的指定位置，瞄准方向任意指定。要求启动瞄准模块在 4s 内自动瞄准发射激光击中靶心，要求 $D1 \leq 2cm$ 。

表 B-3 基础部分 3 测试结果

次数	1	2	3	4	5
打靶用	2.6	3.2	1.3	3.5	2.9

④ 将小车放置在 AB 段轨迹上，前沿投影与 AC 线对齐，启动小车和瞄准模块，在 t 秒内沿轨迹自动寻迹行驶 N 圈，运动期间激光笔必须连续发光射向靶面，不满足要求不计成绩。要求 $N=1$ 圈， $t \leq 20s$ ， $D1 \leq 2cm$ 。

表 B-4 发挥部分 1 测试结果

次数	1	2	3	4	5
寻迹用时 (s)	18	18	18	19	18
最大距离	1.2	1.5	1.7	1.8	1.3

⑤要求 $N=2$ 圈, $t \leq 40s$, $D1 \leq 2cm$ 。

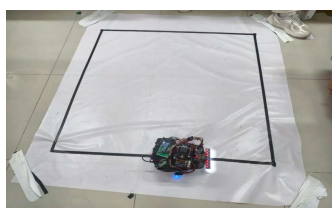
表 B-5 发挥部分 2 测试结果

次数	1	2	3	4	5
寻迹用时(s)	36	37	38	37	37
最大距离	1.9	1.7	1.8	1.8	1.6

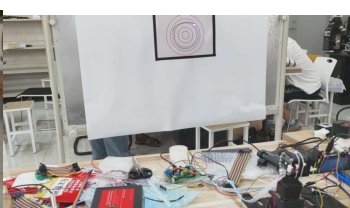
⑥要求 $N=1$ 圈, $t \leq 20s$, 激光笔沿靶面上半径 6cm 的红色圆弧同步画圆, 光斑痕迹与半径 6cm 的红色圆弧线最大距离 $D2 \leq 2cm$; 小车行驶 1 圈, 正好画 1 圈光斑, 同步误差必须小于 $1/4$ 圈。

表 B-6 发挥部分 3 测试结果

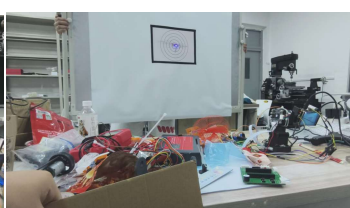
次数	1	2	3	4	5
寻迹用时(s)	18	17	19	16	18
最大距离	2.0	1.6	1.7	1.5	1.9
同步误差	1/8	1/8	1/7	1/8	1/7



图三



图四



图五

2、测试分析和结论

根据上述测试数据, 可以得出以下结论:

- (1) 能够很好的完成基本功能和发挥功能。
- (2) 完成任务的时间较快。

综上所述, 本设计基本达到设计要求。

第五章 结论与心得

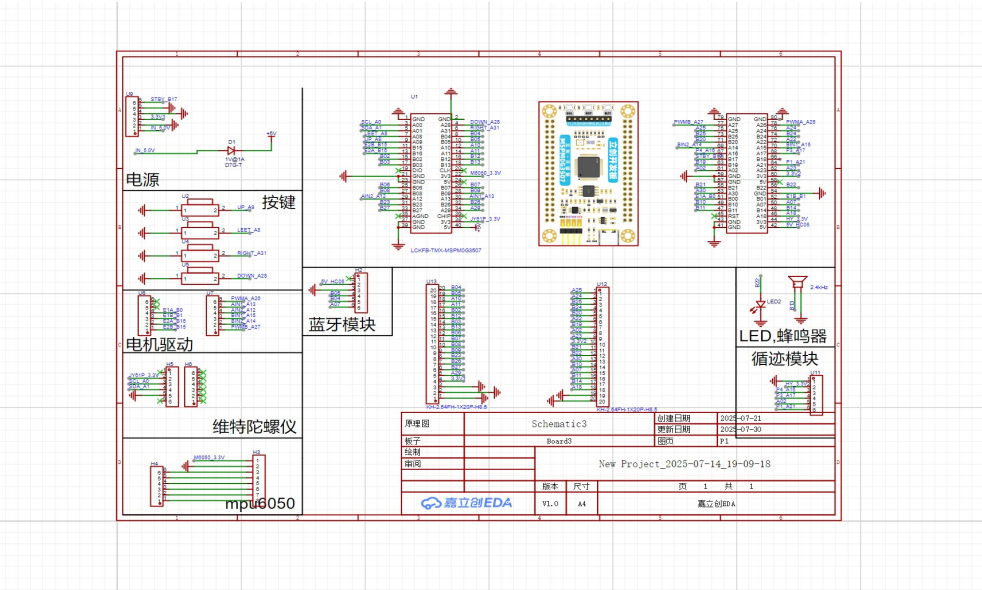
本设计完成了一个基于 MSPM0G3507 的简易自瞄准装置，实现了自动循迹、激光瞄准、动态同步等功能，满足了题目的要求。通过优化灰度传感器布局 and PID 参数整定，提高了小车在复杂路径下的跟踪稳定性。在此过程中，团队解决了跟踪稳定性、泛倾斜抗干扰等问题，提高了嵌入式开发和实时控制的技术能力。动态画圆同步误差控制在 1/6 圆以内，满足实时跟踪要求。这些实践加深了对嵌入式系统和控制算法的理解，也增强了团队分阶段解决问题的协作能力。

参考文献

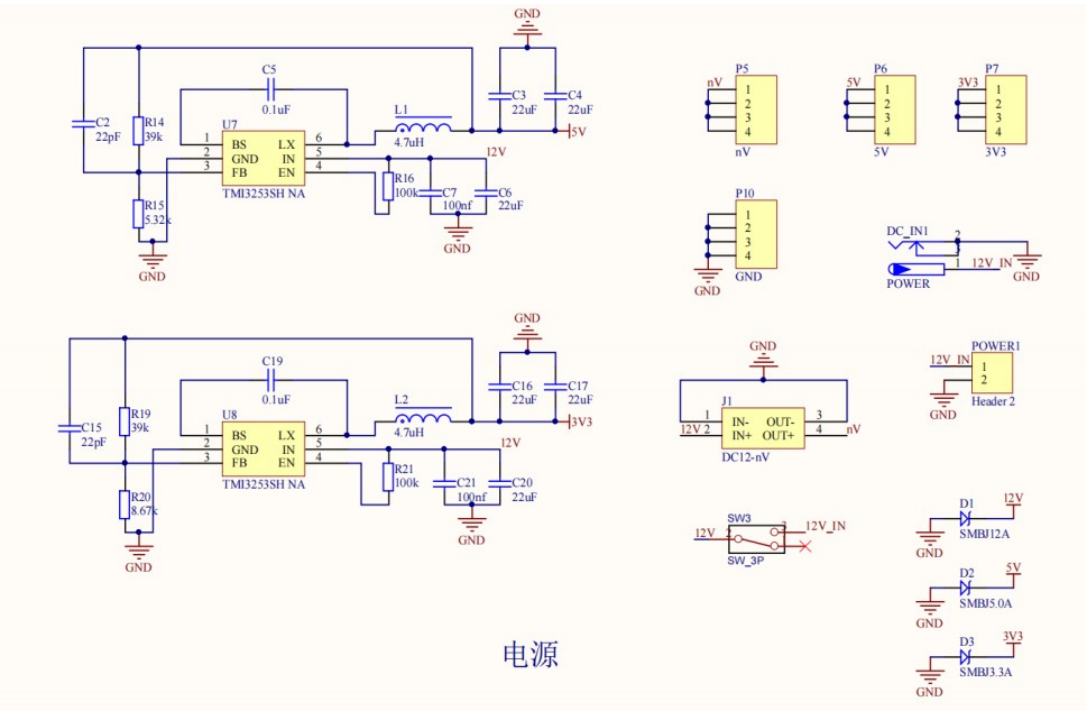
- [1]陈佳琪,许天乐,陈雅芝,等. 基于 MSPM0G3507 微控制器的智能小车的设计 [J]. 工业控制计算机, 2025, 38 (06): 157-158+161. [2]黄智伟. 全国大学生电子设计竞赛训练教程. 北京: 电子工业出版社, 2005.
- [2]陈佳琪,许天乐,陈雅芝,等. 基于 MSPM0G3507 微控制器的智能小车的设计 [J]. 工业控制计算机, 2025, 38 (06): 157-158+161.
- [3]邱晨曦. 循迹避障智能小车设计及分析 [J]. 专用汽车, 2025, (07): 36-41. DOI:10.19999/j.cnki.1004-0226.2025.07.011.
- [4]童诗白. 模拟电子技术基础 第三版. 北京: 高等教育出版社, 2001. 1.

附录一：电路原理图

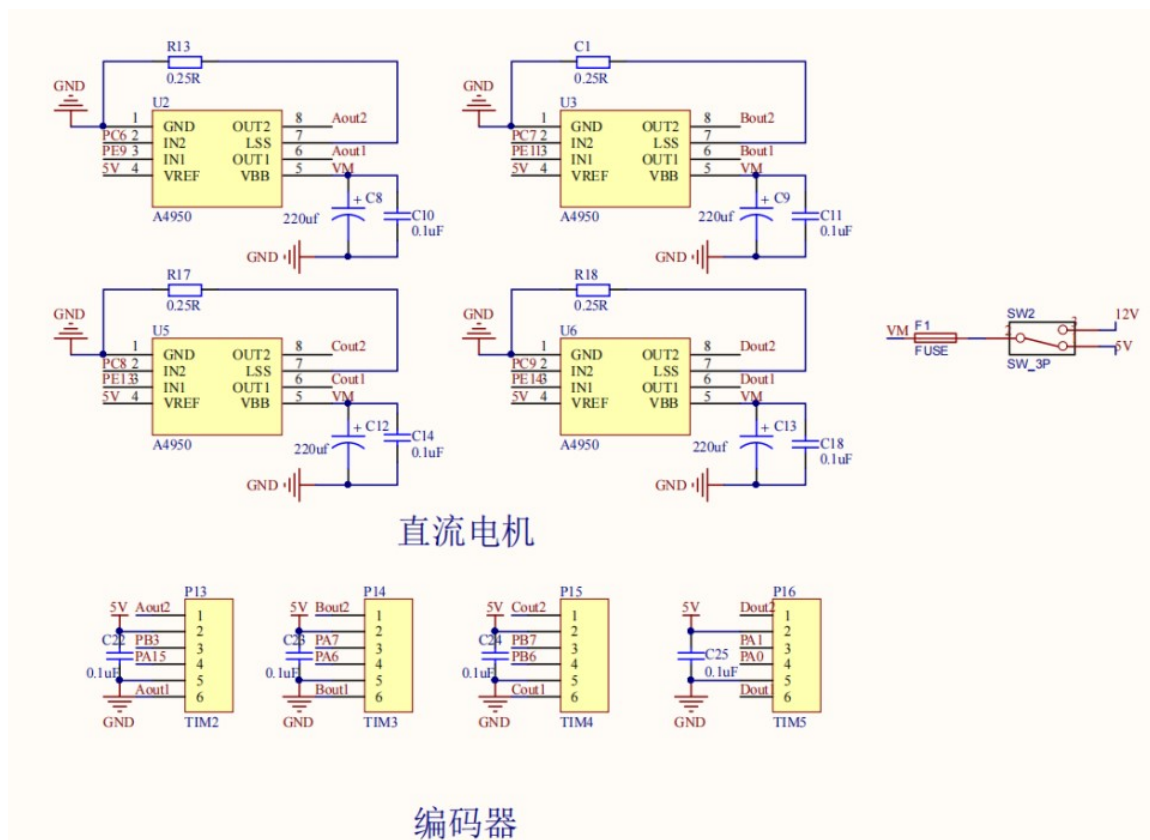
(1) 系统总体电路设计



(2) 电源电路设计



(3) 电机驱动电路设计



附录二：部分源程序

```

import time, os, sys

import math

import cv_lite

import ulab.numpy as np


from machine import UART, FPIOA, Pin
from media.sensor import *
from media.display import *
from media.media import *


#
=====

=====

# --- 1. 核心驱动类: StepperMotorController (无变动) ---
#
=====

=====

class StepperMotorController:
    """ K230 通过 UART 控制 TJ-42Emm 步进闭环驱动器的驱动类 """
    def __init__(self, uart_id, tx_pin, rx_pin, motor_address, baud_rate=115200):
        self.motor_address = motor_address
        fpioa = FPIOA()
        if uart_id == UART.UART2:
            fpioa.set_function(tx_pin, FPIOA.UART2_TXD);
fpioa.set_function(rx_pin, FPIOA.UART2_RXD)
            elif uart_id == UART.UART3:
                fpioa.set_function(tx_pin, FPIOA.UART3_TXD);
fpioa.set_function(rx_pin, FPIOA.UART3_RXD)
            else: raise ValueError("当前代码仅支持 UART2 和 UART3")
        self.uart = UART(uart_id, baud_rate, bits=UART.EIGHTBITS,

```

